Caxi — Como funciona a arquitetura

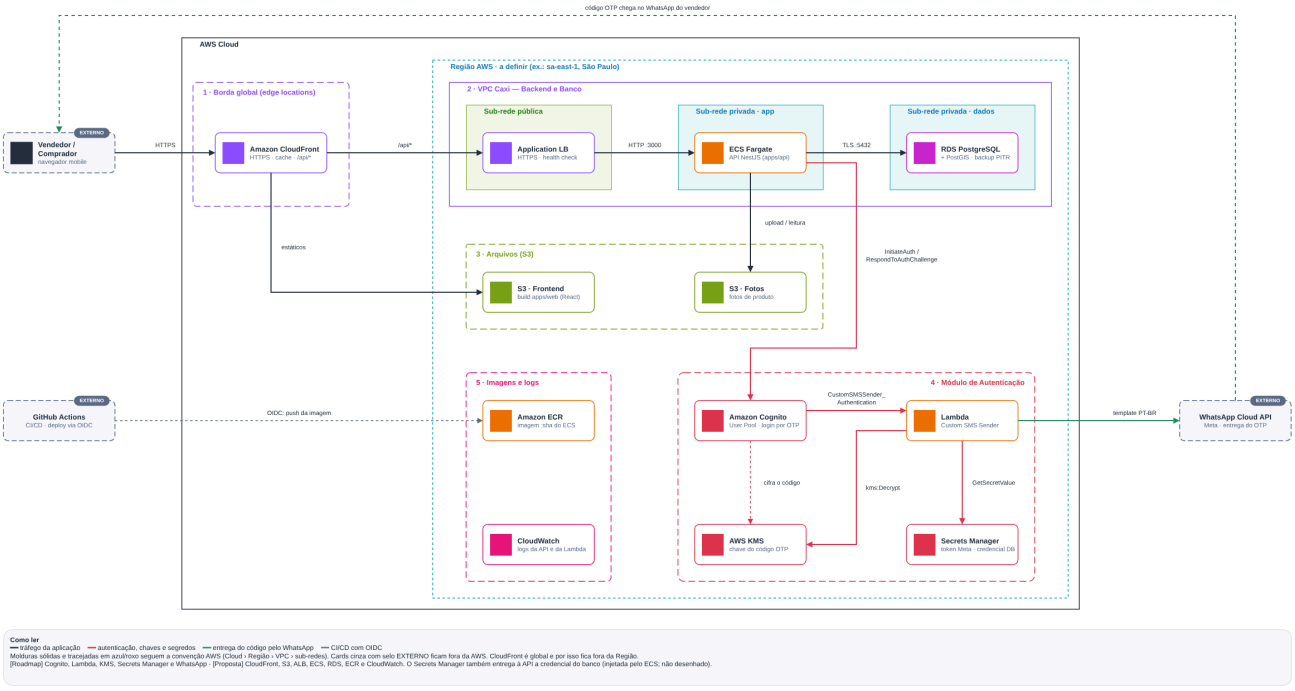
Oct 23, 2026 · @Enzo Rodrigues

1. Visão geral

O Caxi segue uma arquitetura em **três camadas** (apresentação, aplicação e dados) sobre serviços gerenciados da AWS, com a identidade delegada ao Amazon Cognito.

- **Apresentação:** SPA React (Vite), servida como conteúdo estático pelo S3 atrás do CloudFront.
- **Aplicação:** monólito modular em NestJS, executado como container no ECS Fargate atrás de um Application Load Balancer.
- **Dados:** PostgreSQL com PostGIS no RDS, mais S3 para arquivos.

Todo tráfego externo entra por **um único ponto**, o CloudFront. A comunicação interna segue uma cadeia de confiança em que cada camada só aceita conexões da anterior: CloudFront → ALB → ECS → RDS.



Prévia do arquivo `caxi-arquitetura.drawio`. No draw.io, os quadrados coloridos aparecem como os ícones oficiais da AWS.

1.1 Decisões estruturais

Decisão	Escolha	Alternativa descartada	Justificativa
Estilo do backend	Monólito modular NestJS: módulos `acesso`, `barracas`, `produtos`, `pedidos` e `resultados`	Microsserviços	As jornadas compartilham um modelo de dados e precisam de transações entre pedido e estoque. Para um time pequeno, o custo de rede, deploy e observabilidade de vários serviços não se paga no MVP
Entrega do frontend	SPA estática em S3 + CloudFront	Servidor Node para renderizar o front	Nenhum servidor a operar para o front, cache na borda e custo próximo de zero. Stack (React + Vite) fixada no Roadmap
Computação da API	ECS Fargate	EC2 com Docker; AWS Lambda	O Fargate executa a mesma imagem do ambiente local, sem gerenciar servidores. Lambda exigiria adaptar o NestJS e lidar com *cold start* e com o pool de conexões ao PostgreSQL
Banco	RDS for PostgreSQL + PostGIS	PostgreSQL em EC2; Aurora	PostgreSQL + PostGIS fixados no Roadmap. O RDS entrega backup contínuo (PITR), patches e criptografia gerenciados. O Aurora custa mais para o volume do piloto

Decisão	Escolha	Alternativa descartada	Justificativa
Identidade	Cognito User Pools + Lambda Custom SMS Sender	Autenticação própria no NestJS	Fixado no Roadmap. O Cognito já entrega a emissão e assinatura de JWT, a expiração do código e o limite de tentativas
Domínio	SPA e API no mesmo domínio, roteadas por caminho no CloudFront	API num subdomínio, com CORS	O cookie de sessão fica *first-party*, e não há política de CORS a manter
Infraestrutura como código	AWS CDK em TypeScript	Terraform	Mesma linguagem do resto do projeto, com tipagem e revisão por PR

1.2 Hierarquia de rede e de conta

Nível	Recurso	O que define
AWS Cloud	Conta AWS	Limite de faturamento e de IAM. Proposta: piloto e produção em contas separadas
Borda global	CloudFront + certificado ACM	Terminação TLS e cache nos pontos de presença. O certificado do CloudFront precisa ficar em `us-east-1`, exigência do serviço
Região	`sa-east-1` (proposta)	Onde ficam todos os recursos regionais. Afeta latência, custo e a localização dos dados pessoais (LGPD)
VPC	Rede privada, em pelo menos 2 zonas de disponibilidade	O ALB exige sub-redes em 2 AZs. O *DB subnet group* do RDS também

Nível	Recurso	O que define
Sub-rede pública	Rota para o Internet Gateway	Apenas o ALB
Sub-rede privada · app	Sem IP público. Saída por NAT Gateway ou VPC endpoints	Tarefas ECS. Precisam alcançar ECR, Secrets Manager, CloudWatch Logs, S3 e Cognito
Sub-rede privada · dados	Sem rota para a internet	Apenas o RDS

No diagrama, as setas **pretas** são tráfego da aplicação, as **vermelhas** são autenticação, chaves e segredos, as **verdes** são a entrega do OTP e as **cinza tracejadas** são CI/CD.

2. Fluxo de requisição: carregamento do catálogo

Um comprador abre o link público de uma barraca. O fluxo tem duas fases: o navegador baixa a SPA, e depois a SPA consulta a API. No diagrama, são as setas pretas.

2.1 Fase estática: entrega da SPA

1. **Resolução e TLS.** O domínio do Caxi aponta, por DNS, para a distribuição do CloudFront. A conexão TLS termina no ponto de presença mais próximo do comprador, com o certificado do ACM.
2. **Comportamento padrão (`/*`).** O CloudFront consulta o cache. Num *miss*, busca o objeto na origem S3 Frontend, autenticando-se por **Origin Access Control (OAC)**. O bucket é privado e sua *bucket policy* só aceita leituras assinadas por essa distribuição.
3. **Cache.** Os arquivos de `assets/` têm *hash* no nome, gerado pelo build do Vite, e podem ficar em cache por longos períodos. O `index.html` muda a cada deploy e por isso é invalidado no pipeline.
4. **Rotas da SPA.** Caminhos como `/b/demo` não existem no bucket. O CloudFront responde com o `index.html`, e o roteador do React resolve a rota no navegador. Localmente, o `try_files` do nginx cumpre esse papel.

2.2 Fase dinâmica: chamada à API

1. **Requisição.** A SPA chama `GET /api/barracas/demo` no **mesmo domínio**.

2. **Comportamento `/api/*`.** O CloudFront encaminha para a origem ALB **sem cache** e repassa cookies, *query string* e os cabeçalhos necessários. Em produção, um cabeçalho secreto ou a *prefix list* gerenciada do CloudFront impede que o ALB seja chamado diretamente, contornando a borda (proposta).
3. **ALB.** O *listener* HTTPS (443) encaminha para um *target group* do tipo `ip`, exigido pelo modo de rede `awsvpc` do Fargate, na porta 3000. O ALB só envia tráfego a tarefas que passaram no health check `GET /api/health/ready`.
4. **ECS Fargate.** Cada tarefa tem uma ENI própria na sub-rede privada. O *security group* da tarefa aceita a porta 3000 **apenas a partir do security group do ALB**.
5. **NestJS.** A requisição atravessa o pipeline do framework: *middleware* de log (com `requestId` e *redaction*) → *guards* de autenticação e autorização, nas rotas privadas → *pipes* de validação → *controller* → *service* do módulo `barracas` → repositório TypeORM.
6. **RDS.** O TypeORM usa um pool de conexões com TLS na porta 5432. O security group do RDS aceita 5432 **apenas a partir do security group das tarefas**. As credenciais chegam à tarefa como variáveis injetadas pelo ECS a partir do Secrets Manager.
7. **Resposta.** JSON pelo caminho inverso, sem cache no CloudFront.

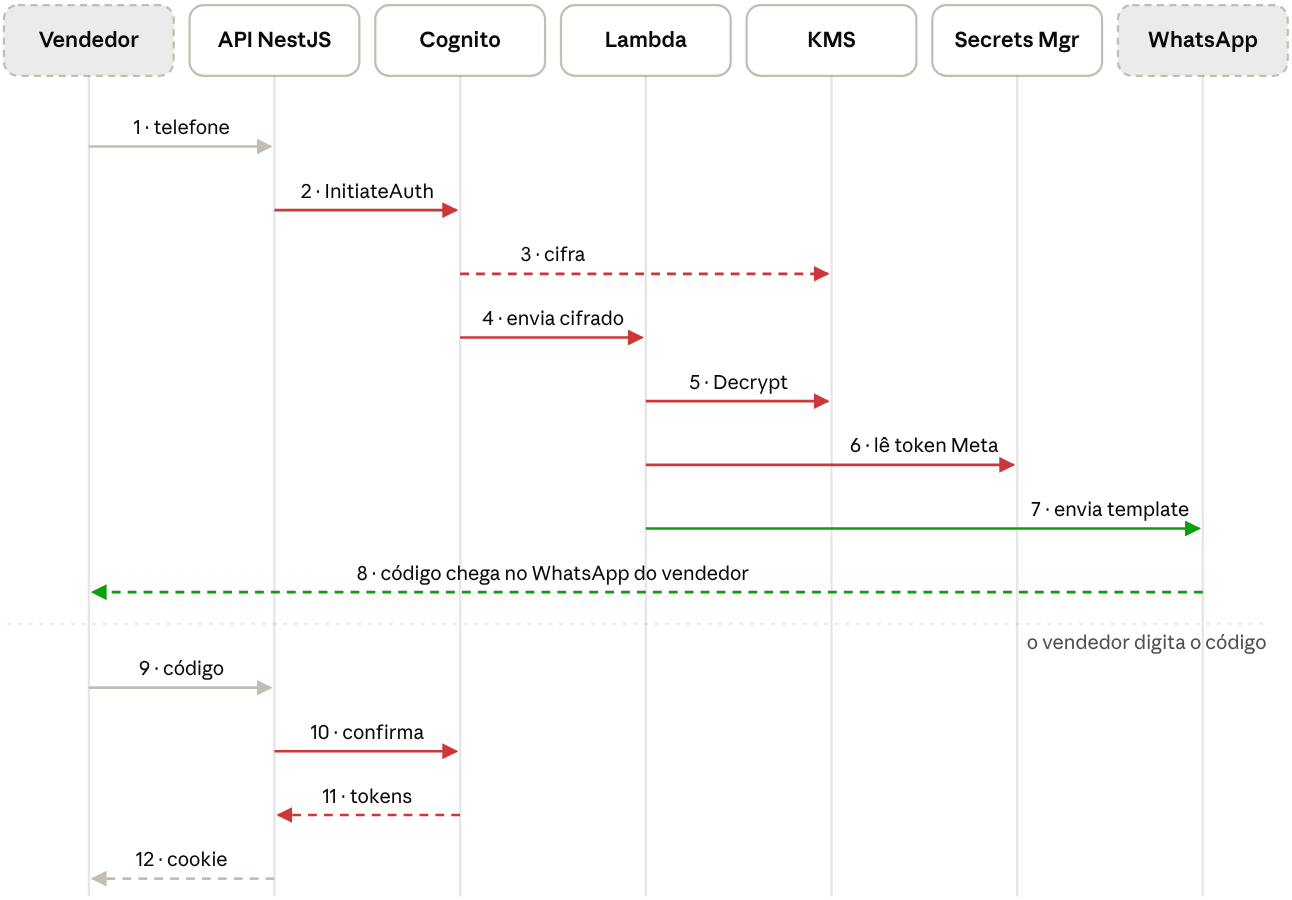
Por que o mesmo domínio para SPA e API? O navegador trata as chamadas como *same-origin*. Isso permite um cookie de sessão `HttpOnly; Secure; SameSite=Lax` sem política de CORS. Uma API num subdomínio exigiria CORS com credenciais, uma configuração comum de dar errado.

Fotos de produto. A API grava os arquivos no bucket S3 Fotos pela API do S3, com permissão restrita ao prefixo do bucket. A entrega ao navegador ainda está em aberto: URL pré-assinada com expiração curta, ou uma segunda origem no CloudFront, também com OAC.

3. Fluxo de autenticação: login sem senha com OTP no WhatsApp

O vendedor não tem senha. O Cognito usa o fluxo nativo *passwordless* `USER_AUTH` com o desafio `SMS_OTP`, que exige o plano Essentials ou Plus. A entrega por SMS é substituída pelo gatilho **Custom SMS Sender**: o Cognito entrega o código cifrado a uma Lambda nossa, que o envia pelo WhatsApp. No diagrama, são as setas vermelhas e verdes.

O código nasce no Cognito, viaja cifrado e só é aberto na Lambda



login sem senha · 12 passos

Passo	Origem → destino	Chamada e dados	Observação técnica
1	SPA → API	`POST /api/auth/otp` com o telefone em E.164	O módulo `acesso` confere se o convite foi aprovado. Respostas iguais para telefone existente ou não, para não revelar cadastros
2	API → Cognito	`InitiateAuth`: `AuthFlow=USER_AUTH`, `AuthParameters={USERNAME, PREFERRED_CHALLENGE=SMS_OTP, SECRET_HASH}`	O app client é confidencial (tem *client secret*), então toda chamada leva o `SECRET_HASH` (HMAC do usuário com o segredo). A resposta traz `ChallengeName=SMS_OTP` e um `Session` opaco, que a API guarda para o passo 10

Passo	Origem → destino	Chamada e dados	Observação técnica
3	Cognito → KMS	Cifragem do código com a chave simétrica configurada no user pool (`CustomSenderKmsKey`)	O código em texto puro existe só dentro do Cognito. A *key policy* permite ao Cognito cifrar
4	Cognito → Lambda	Invocação com `triggerSource=CustomSMSSender_Authentication` ; `event.request.code` = código cifrado em base64	A Lambda tem *resource policy* permitindo a invocação pelo `cognito-idp.amazonaws.com` daquele pool
5	Lambda → KMS	`decrypt` do AWS Encryption SDK com `KmsKeyringNode` , que chama `kms:Decrypt`	**Só a role da Lambda tem `kms:Decrypt`** nessa chave. O texto decifrado vive apenas em memória, durante a invocação
6	Lambda → Secrets Manager	`GetSecretValue` pelo ARN exato do segredo da Meta	Valor em cache no escopo global da função, reaproveitado entre invocações da mesma instância (menos latência e custo)
7	Lambda → Meta	`POST https://graph.facebook.com/<versão>/<phone-number-id>/messages` com um template de categoria *authentication* em PT-BR	Logs registram só status HTTP, `messageId` e código de erro. Nunca o `event` , o código ou o corpo da requisição
8	Meta → vendedor	Mensagem no WhatsApp	Fora da AWS. Falhas de entrega aparecem como erro na resposta da Graph API e em métricas da Lambda
9	SPA → API	`POST /api/auth/otp/confirm` com o código	Validação de formato no NestJS antes de chamar o Cognito
10	API → Cognito	`RespondToAuthChallenge` : `ChallengeName=SMS_OTP` , `ChallengeResponses={USERNAME, SMS_OTP_CODE, SECRET_HASH}` e o `Session` do passo 2	O Cognito aplica expiração do código e limite de tentativas

Passo	Origem → destino	Chamada e dados	Observação técnica
11	Cognito → API	`AuthenticationResult`: `IdToken`, `AccessToken`, `RefreshToken`	Os JWT são assinados com RS256. A API valida assinatura (JWKS do pool), `iss`, `token_use`, `client_id` e `exp`
12	API → SPA	`Set-Cookie` de sessão `HttpOnly; Secure; SameSite=Lax`	Os tokens ficam no servidor; o navegador recebe só o identificador da sessão, com proteção CSRF. A API associa o `sub` do Cognito ao proprietário no banco

3.1 Sessão depois do login

- **Autorização:** o Cognito diz **quem** é o usuário (`sub`). **O que** ele pode fazer, como editar uma barraca, é decidido pelos *guards* do NestJS a partir do vínculo `sub` → proprietário → barraca no PostgreSQL.
- **Renovação:** quando o `AccessToken` expira, a API usa o `RefreshToken` (`InitiateAuth` com `REFRESH_TOKEN_AUTH`) sem envolver o navegador.
- **Saída:** a API revoga o refresh token no Cognito (`RevokeToken`) e destrói a sessão.
- **Em aberto:** onde ficam as sessões do lado do servidor (tabela no PostgreSQL ou cache em memória compartilhado) e a duração de cada token.

Por que não enviar o código direto da API? O Cognito não tem integração nativa com WhatsApp, só SMS e e-mail. O Custom SMS Sender é o ponto de extensão oficial para trocar o canal sem perder o que o Cognito já controla: geração, expiração e verificação do código, limite de tentativas e emissão de tokens. A cifragem com KMS garante que o código nunca trafegue em texto puro entre serviços da AWS.

4. Componentes: papel e configuração

Cada componente tem um trabalho só. A regra que guiou as escolhas: **serviço gerenciado sempre que possível**.

A configuração abaixo é a **proposta** para o provisionamento em AWS CDK. Nada foi provisionado ainda (seção 7). Valores numéricos de capacidade, como CPU, memória e classe de instância, ficam para quando houver medição de uso.

4.1 Externos

Componente	Papel	Configuração e integração
Navegador (vendedor e comprador)	Executa a SPA	Guarda apenas o cookie de sessão `HttpOnly`. Nenhum token fica acessível ao JavaScript
GitHub Actions	CI e, na proposta, CD	CI: checks `qualidade`, `ambiente-reproduzivel` e `padroes-do-pr`. CD: assume uma role IAM por **OIDC** (`token.actions.githubusercontent.com`), com *trust policy* restrita a `repo:Rodriguessz/caxi:environment:<ambiente>`. Credenciais temporárias, nenhuma chave guardada no GitHub
WhatsApp Cloud API (Meta)	Entrega do OTP	Graph API com número remetente e template de categoria *authentication* aprovados. Token de acesso de *system user* guardado no Secrets Manager. Número, template e token separados por ambiente

4.2 Borda global

Componente	Papel	Configuração proposta
Amazon CloudFront	Ponto único de entrada, TLS e cache	**Origens:** S3 Frontend (via OAC) e ALB (HTTPS). **Comportamentos:** padrão `/*` com política de cache otimizada; `/api/*` com `CachingDisabled` e *origin request policy* que repassa cookies, *query string* e cabeçalhos. *Viewer protocol* `redirect-to-https`, TLS 1.2 ou superior, certificado ACM em `us-east-1`. **Rotas da SPA:** reescrever caminhos sem extensão para `/index.html` com uma CloudFront Function no comportamento padrão. *Custom error responses* (403/404 → `index.html`) valem para a distribuição inteira e transformariam os 404 da API em HTML

4.3 VPC: backend e banco

Componente	Papel	Configuração proposta
Application Load Balancer	Balanceamento e health check da API	*Internet-facing*, em sub-redes públicas de 2 AZs. *Listener* 443 com certificado ACM regional e 80 → 443. *Target group* tipo `ip`, porta 3000, health check `/api/health/ready`. *Security group*: entrada 443 só da *prefix list* gerenciada `com.amazonaws.global.cloudfront.origin-facing`; saída 3000 só para o SG das tarefas

Componente	Papel	Configuração proposta
ECS Fargate	Executa a API NestJS	*Service* com 2 tarefas ou mais, em AZs diferentes (no piloto, 1 tarefa se o custo pesar). *Task definition*: imagem `caxi-api:<sha>` do ECR, porta 3000, configuração não sensível em `environment`, credenciais em `secrets` (Secrets Manager), logs pelo driver `awslogs`, health check do container em `/api/health/live`. Deploy *rolling* com **deployment circuit breaker** e rollback automático. Migrations rodam como tarefa avulsa (`RunTask`) com a mesma imagem, antes de atualizar o service
RDS for PostgreSQL	Persistência transacional e geoespacial	PostgreSQL 16 com a extensão PostGIS, criada pela primeira migration (`CREATE EXTENSION postgis`). `PubliclyAccessible=false`, *DB subnet group* nas sub-redes de dados. Armazenamento cifrado com KMS. TLS obrigatório (`rds.force_ssl=1`). Backups automáticos com PITR (retenção sugerida de 7 dias), *deletion protection* e senha mestre gerenciada pelo RDS no Secrets Manager. Multi-AZ opcional, por custo

A task definition do ECS separa duas roles. A **execution role** é usada pelo agente do ECS para baixar a imagem do ECR, ler os segredos e escrever logs. A **task role** é usada pelo código da API para chamar Cognito e S3. Assim, o código da aplicação não recebe permissões de infraestrutura.

4.4 Arquivos (S3)

Componente	Papel	Configuração proposta
S3 · Frontend	Artefatos do build do Vite	*Block Public Access* ligado. *Bucket policy* que só aceita `s3:GetObject` da distribuição via OAC. Versionamento ligado: o build anterior continua disponível para rollback. Publicado pelo pipeline com `s3 sync` + invalidação do `index.html`

Componente	Papel	Configuração proposta
S3 · Fotos	Fotos de produto	*Block Public Access* ligado, criptografia em repouso e versionamento contra sobrescrita e exclusão acidentais. Escrita e leitura só pela *task role* da API. *Lifecycle rule* para expirar versões antigas

4.5 Módulo de autenticação

Componente	Papel	Configuração proposta
Amazon Cognito User Pool	Identidade, desafio OTP e emissão de tokens	Plano Essentials. Login pelo atributo `phone_number`. *Self sign-up* desligado: a API cria o usuário (`AdminCreateUser`) quando o convite é aprovado. Primeiro fator permitido: `SMS_OTP`. *App client* confidencial com `ALLOW_USER_AUTH` e `ALLOW_REFRESH_TOKEN_AUTH`. `LambdaConfig` com `CustomSMSSender` e a chave KMS (`KMSKeyID`)
Lambda Custom SMS Sender	Decifra e entrega o OTP	Runtime Node.js, fora da VPC: só acessa endpoints públicos da AWS e da Meta. Variáveis `KEY_ARN` e `META_SECRET_ARN`. *Resource policy* permitindo invocação pelo Cognito daquele pool. Retenção de logs definida. Alarmes de `Errors` e `Throttles`
AWS KMS	Chave do OTP	Chave simétrica gerenciada pelo cliente, uma por ambiente, com rotação automática. A *key policy* permite ao Cognito usar a chave para cifrar. `kms:Decrypt` só para a role da Lambda

Componente	Papel	Configuração proposta
Secrets Manager	Segredos de aplicação	`caxi/<env>/meta-whatsapp` (só a role da Lambda lê), segredo gerenciado do RDS com rotação, *client secret* do Cognito e chave de sessão (só a execution role da API lê). Acesso por ARN exato, nunca por curinga

4.6 Imagens e logs

Componente	Papel	Configuração proposta
Amazon ECR	Registro das imagens da API	Tags **imutáveis**: cada imagem é `caxi-api:<sha do commit>` e não pode ser sobrescrita. *Scan on push* ligado. *Lifecycle policy* mantendo as últimas versões, para rollback
Amazon CloudWatch	Logs, métricas e alarmes	*Log groups* da API e da Lambda com retenção definida (sugestão: 30 dias no piloto). Logs em JSON, com `requestId` e *redaction*. Alarmes via SNS: 5xx e *unhealthy hosts* no ALB, reinícios de tarefa no ECS, CPU, armazenamento e conexões do RDS, erros e *throttles* da Lambda. CloudTrail ligado para auditoria

5. Matriz de comunicação

Cada conexão do diagrama é controlada em duas camadas: **rede** (*security groups*, rotas, *bucket policies*) e **identidade** (IAM, OAC, tokens). Tudo o que não aparece nesta tabela é negado por padrão.

Origem → destino	Protocolo / porta	Autenticação e autorização	Controle de rede
Navegador → CloudFront	HTTPS 443 (TLS 1.2+)	Rotas públicas: nenhuma. Rotas privadas: cookie de sessão, validado na API	Público, só HTTPS (`redirect-to-https`)
CloudFront → S3 Frontend	HTTPS (API do S3)	OAC: requisições assinadas (SigV4) pela distribuição	*Bucket policy* aceita só aquela distribuição; *Block Public Access* ligado
CloudFront → ALB	HTTPS 443	Cabeçalho secreto de origem (proposta)	SG do ALB aceita só a *prefix list* do CloudFront
ALB → ECS Fargate	HTTP 3000 (dentro da VPC)	—	SG das tarefas aceita 3000 só do SG do ALB
ECS → RDS	PostgreSQL 5432 com TLS	Usuário e senha do banco, injetados pelo ECS a partir do Secrets Manager	SG do RDS aceita 5432 só do SG das tarefas; sub-rede sem rota para a internet
ECS → S3 Fotos	HTTPS (API do S3)	*Task role*: `s3:GetObject` e `s3:PutObject` só nesse bucket	Saída por NAT Gateway ou *gateway endpoint* do S3
ECS → Cognito	HTTPS (API `cognito-idp`)	`SECRET_HASH` do app client; *task role* com as ações necessárias só nesse user pool	Saída pela sub-rede privada
ECS → Secrets Manager / ECR / CloudWatch	HTTPS	*Execution role*, com acesso a ARNs específicos	Saída por NAT Gateway ou VPC endpoints

Origem → destino	Protocolo / porta	Autenticação e autorização	Controle de rede
Cognito → KMS	Interno da AWS	*Key policy* permite ao Cognito cifrar com a chave	—
Cognito → Lambda	Invocação de gatilho	*Resource policy* da Lambda restrita ao `cognito-idp.amazonaws.com` daquele user pool	—
Lambda → KMS	HTTPS (API do KMS)	Role da Lambda: `kms:Decrypt` só nessa chave	Lambda fora da VPC
Lambda → Secrets Manager	HTTPS	Role da Lambda: `secretsmanager:GetSecretValue` só no ARN do segredo da Meta	Lambda fora da VPC
Lambda → WhatsApp Cloud API	HTTPS 443	Token Bearer da Meta	Internet pública
GitHub Actions → ECR (e ECS, S3, CloudFront no CD)	HTTPS	OIDC → `sts:AssumeRoleWithWebIdentity`, com role restrita ao repositório e ao ambiente	Internet pública
ECS e Lambda → CloudWatch Logs	HTTPS	Execution role (ECS) e role da Lambda, com escrita só nos próprios *log groups*	—

6. Segurança e resiliência

A arquitetura aplica **defesa em profundidade**: cada camada tem seu próprio controle, então a falha de um deles não expõe as outras.

6.1 Controles por camada

Camada	Controle	Ameaça mitigada
Borda	Ponto único de entrada (CloudFront), HTTPS obrigatório, WAF com limite de taxa no login (opcional no piloto)	Acesso direto à origem, interceptação, abuso do endpoint de OTP
Rede	Sub-redes privadas e cadeia de *security groups* (CloudFront → ALB → ECS → RDS)	Movimentação lateral; banco alcançável pela internet
Identidade	Cognito para autenticação, *guards* do NestJS para autorização por barraca	Acesso a dados de outro vendedor, também validado nos testes do Roadmap
Sessão	Tokens só no servidor; cookie `HttpOnly; Secure; SameSite=Lax` com token CSRF	Roubo de token por XSS; requisições forjadas
OTP	Cifrado com KMS de ponta a ponta entre serviços, decifrado só na Lambda	Vazamento do código em eventos, filas ou logs
IAM	Uma role por componente, com ações e ARNs específicos; execution role separada da task role; OIDC no CI	Escalada de privilégio; vazamento de credencial de longa duração
Segredos	Secrets Manager, injetados em tempo de execução; gitleaks no CI	Segredo no repositório, na imagem Docker ou no log do CI
Dados em repouso	RDS, S3 e segredos cifrados com KMS	Exposição de mídia ou snapshot
Dados em trânsito	TLS em todos os saltos, inclusive até o banco (`rds.force_ssl`)	Leitura do tráfego interno
Auditoria	CloudTrail e logs estruturados com `requestId`	Falta de rastreabilidade num incidente

6.2 Dados pessoais (LGPD)

- **Dados tratados:** telefone e nome do vendedor, dados da barraca, itens e contato dos pedidos.
- **Minimização nos logs:** *redaction* obrigatória de `authorization`, `cookie`, `set-cookie`, `code`, `otp` e tokens; telefone mascarado (`+55 11 *****-1234`). A Lambda nunca registra o `event`, o código nem o corpo da chamada à Meta. Um teste automatizado deve falhar se algum log contiver o OTP ou um JWT.
- **Localização:** com a Região `sa-east-1`, os dados ficam armazenados no Brasil. A Meta processa o número de telefone e o conteúdo do template para entregar a mensagem.
- **Separação:** dados de teste e de uso real ficam em ambientes separados e identificados, como exige o Roadmap.

6.3 Resiliência e recuperação

Cenário	Mecanismo	Meta sugerida para o piloto
Falha de uma tarefa da API	Health check do ALB retira a tarefa; o ECS sobe outra	Sem intervenção manual
Deploy com defeito	*Deployment circuit breaker* volta à task definition anterior; imagem anterior no ECR por SHA	Rollback automático
Front com defeito	Republicar o build anterior, mantido versionado no S3, e invalidar o CloudFront	Minutos
Migration com defeito	Padrão *expand/contract* e `down()` testado no CI; snapshot do RDS antes de cada migration	Reverter sem perda de dados
Dados corrompidos ou apagados	Restauração do RDS para um ponto no tempo (PITR)	RPO ⩽ 5 min; RTO ⩽ 1 h
Perda de uma zona de disponibilidade	ALB e tarefas em 2 AZs; RDS Multi-AZ (opcional)	Continuidade da API; banco depende do Multi-AZ

7. Decidido, proposto e em aberto

O Roadmap da Sprint 2 fixou a stack e o módulo de autenticação. A hospedagem é proposta deste plano. Nada da AWS foi provisionado ainda: hoje roda apenas o ambiente local e o CI no GitHub.

Situação	Componentes
Decidido no Roadmap	React, NestJS, PostgreSQL + PostGIS (TypeORM), Cognito, Lambda Custom SMS Sender, KMS, Secrets Manager, WhatsApp Cloud API
Proposto neste plano	CloudFront, S3 (site e fotos), ALB, ECS Fargate, RDS, ECR, CloudWatch, deploy por GitHub Actions com OIDC
Já funcionando	Ambiente local com Docker Compose e CI no GitHub com proteção da `main` (ver o guia do ambiente reproduzível)

Em aberto antes do piloto:

- [ ] Região AWS. A proposta é São Paulo (`sa-east-1`); falta confirmar se o Cognito oferece login sem senha lá
- [ ] Plano do Cognito: o login sem senha exige o plano Essentials ou Plus
- [ ] Número, conta e template da Meta para o piloto
- [ ] Como o celular baixa as fotos: link assinado ou via CloudFront
- [ ] Custo estimado mensal do piloto, como pede o Roadmap

Alternativa mais barata, se o custo pesar: uma única máquina (EC2 ou Lightsail) rodando o mesmo Docker Compose do ambiente local. Funciona, mas perde o rollback automático, o backup contínuo e a separação de rede descritos aqui.